COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010 | Data Structures and Algorithms	**SECTION:**	CPE21S3
DATE PERFORMED:	Sep 8, 2026	**DATE SUBMITTED:**	Sep 10, 2026
NAME:	Ellaiza C. Vizcayno	**INSTRUCTOR:**	Engr. Jimlord Quejado

ACTIVITY NO. 7 **Sort Techniques**

1. PROCEDURE OUTPUT

Code

```
#include <iostream>
#include <cstdlib>
#include <time.h>
#include "sorting.h"

const int max_size = 100;
void display(int list[]){
    for(int i = 0; i < max_size; i++ ){
        std::cout << list[i] << " ";
        if((i+1) % 10 == 0){
            std::cout << std::endl;
        }
    }
}

int main(){
    int randArr[max_size];
    srand(time(0));

    for(int i = 0; i < max_size; i++){
        randArr[i] = rand()%100;
    }
    display(randArr);

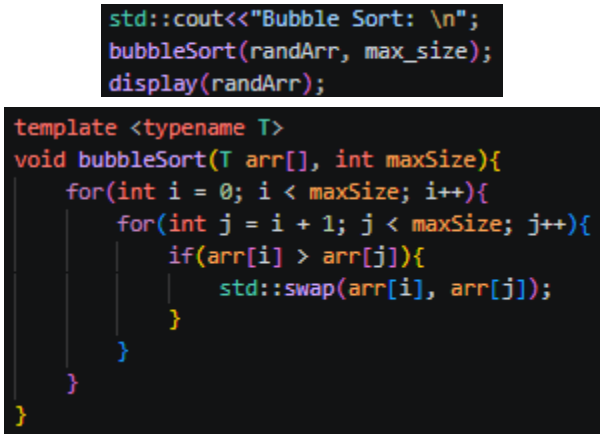
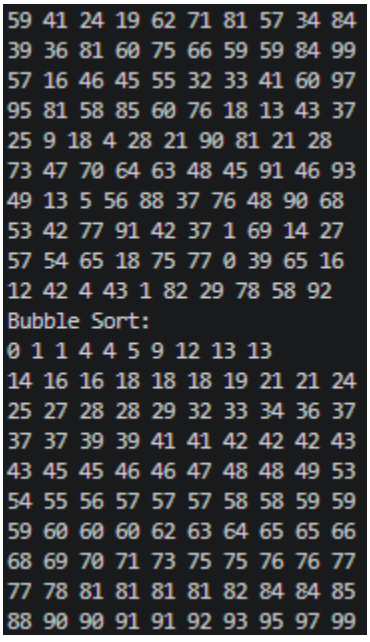
    return 0;
}
```

Console

```
93 35 85 4 10 61 67 32 11 40
4 3 27 48 21 80 32 18 20 76
73 55 53 42 22 33 72 73 28 62
69 62 47 61 40 38 50 13 14 18
74 91 19 9 11 4 61 46 18 19
45 51 0 12 53 97 25 48 30 81
85 71 79 42 19 40 77 84 7 87
44 32 77 1 42 9 38 79 43 90
62 64 37 33 95 53 81 60 26 34
53 90 23 95 71 28 69 90 58 88
```

Observations	This randomly generated 100-element array will be used to test sorting algorithms. The use of the time library helped in achieving random values. Using time means the random values are generated according to the current time, so every time the code is run, the values will always be different. In generating and assigning the random values per index using a for loop, modulo 100 is used to ensure that the values stay under 100 and don't get too high.

Table 7-1. Array of Values for Sort Algorithm Testing

Code	
Console	
Observations	From my understanding, this sorting algorithm is depicted as an air bubble rising to the surface of water, where the highest value gradually moves toward the end of the array. However, in this implementation, each element is compared against every element that comes after it, not just the one directly next to it, and a swap happens whenever the current element is greater than the one it's being compared to.

```
std::cout<<"Bubble Sort: \n";
bubbleSort(randArr, max_size);
display(randArr);
```

```
template <typename T>
void bubbleSort(T arr[], int maxSize){
    for(int i = 0; i < maxSize; i++){
        for(int j = i + 1; j < maxSize; j++){
            if(arr[i] > arr[j]){
                std::swap(arr[i], arr[j]);
            }
        }
    }
}
```

```
59 41 24 19 62 71 81 57 34 84
39 36 81 60 75 66 59 59 84 99
57 16 46 45 55 32 33 41 60 97
95 81 58 85 60 76 18 13 43 37
25 9 18 4 28 21 90 81 21 28
73 47 70 64 63 48 45 91 46 93
49 13 5 56 88 37 76 48 90 68
53 42 77 91 42 37 1 69 14 27
57 54 65 18 75 77 0 39 65 16
12 42 4 43 1 82 29 78 58 92
Bubble Sort:
0 1 1 4 4 5 9 12 13 13
14 16 16 18 18 18 19 21 21 24
25 27 28 28 29 32 33 34 36 37
37 37 39 39 41 41 42 42 42 43
43 45 45 46 46 47 48 48 49 53
54 55 56 57 57 57 58 58 59 59
59 60 60 60 62 63 64 65 65 66
68 69 70 71 73 75 75 76 76 77
77 78 81 81 81 81 82 84 84 85
88 90 90 91 91 92 93 95 97 99
```

Table 7-2. Bubble Sort Technique

Code

```
std::cout<<"Selection Sort: \n";
selectionSort(randArr, max_size);
display(randArr);
```

```
template <typename T>
int routine_smallest(T arr[], int k, int arrSize){
    int position;

    T smallestElement = arr[k];

    for(int j = k + 1; j < arrSize; j++){
        if(arr[j] < smallestElement){
            smallestElement = arr[j];
            position = j;
        }
    }
    return position;
}

template <typename T>
void selectionSort(T arr[], int arrSize){
    int min;
    for(int i = 0; i < arrSize; i++){
        min = routine_smallest(arr, i, arrSize);
        std::swap(arr[i], arr[min]);
    }

}
```

Console

```
37 71 61 88 0 71 14 96 70 38
66 71 46 60 25 4 37 3 98 24
54 18 3 34 40 82 34 29 38 61
98 22 79 77 83 5 87 27 90 70
58 69 81 8 47 25 66 16 24 7
57 94 20 81 30 74 37 93 79 13
36 10 11 26 5 90 16 54 43 55
87 58 3 73 46 76 39 20 89 33
7 63 95 74 23 92 14 0 4 92
21 28 11 1 27 50 46 38 38 1
Selection Sort:
0 0 1 1 3 3 3 4 4 5
5 7 7 8 10 11 11 13 14 14
16 16 18 20 20 21 22 23 24 24
25 25 26 27 27 28 29 30 33 34
34 36 37 37 37 38 38 38 38 39
40 43 46 46 46 47 50 54 54 55
57 60 58 58 61 61 63 66 66 69
70 70 71 71 71 73 74 74 76 77
79 79 81 81 82 83 87 87 88 89
90 90 92 92 93 94 98 98 95 96
```

Observations	When it comes to selection sort, it assumes the first element is the smallest value. It then goes down the list and compares each element to the current "smallest value," which is initially the first value. If it finds a value smaller than the current smallest, the position of the smallest value is updated to the newly found smallest value. Once it reaches the end of the list, it swaps the former smallest value's position with the newly found smallest value's position. Using a helper function like routine_smallest helps find the position of the smallest value. This is likely why it's called "selection" sort, since it selects the smallest value from the unsorted portion of the array during each pass and swaps it into its correct position.

Table 7-3. Selection Sort Algorithm

Code

```
std::cout<<"Insertion Sort: \n";
insertionSort(randArr, max_size);
display(randArr);
```

```
template <typename T>
void insertionSort(T arr[], int arrSize){
    int k = 1, j;
    T temp;
    while(k < arrSize){
        temp = arr[k];
        j = k - 1;
        while(j >= 0 && temp <= arr[j]){
            arr[j + 1] = arr[j];
            j--;
        }
        arr[j + 1] = temp;
        k++;
    }
}
```

Console

```
14 59 82 69 38 15 86 60 52 93
94 69 91 61 95 54 8 86 73 20
70 49 53 53 5 47 93 39 19 84
98 12 91 89 89 73 44 75 78 49
30 1 22 72 51 72 50 47 97 33
80 18 62 99 10 7 4 21 34 28
41 14 48 68 82 39 94 6 12 1
7 86 67 80 61 64 99 55 58 19
7 89 45 42 80 31 18 81 38 51
30 86 8 24 23 69 50 75 64 53
Insertion Sort:
1 1 4 5 6 7 7 7 8 8
10 12 12 14 14 15 18 18 19 19
20 21 22 23 24 28 30 30 31 33
34 38 38 39 39 41 42 44 45 47
47 48 49 49 50 50 51 51 52 53
53 53 54 55 58 59 60 61 61 62
64 64 67 68 69 69 69 70 72 72
73 73 75 75 78 80 80 80 81 82
82 84 86 86 86 86 89 89 89 91
91 93 93 94 94 95 97 98 99 99
```

Observations	This algorithm assumes the first element is already sorted and moves on to the next element. This next element is compared to the elements before it in the sorted section, starting from the one closest to it. If the current element is smaller, the compared element is shifted to the right to make room, and the comparison continues moving backward through the sorted section. Once it reaches a value that is smaller than it, or reaches the start of the array, the element is placed in that position. This process continues until the end of the array, with the sorted section growing by one element each time. This is likely why it's called "insertion" sort, since it works like sorting a hand of playing cards, picking up a new card and comparing it against the already sorted cards, then inserting it into its correct place among them.

Table 7-4. Insertion Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Candidate 1	Bo Dalton Capistrano
Candidate 2	Cornelius Raymon Agustin
Candidate 3	Deja Jayla Bañaga
Candidate 4	Lalla Brielle Yabut
Candidate 5	Franklin Relano Castro

List of Candidates

PROBLEM: Generate an array A[0…100] of unsorted elements, wherein the values in the array are indicative of a vote to a candidate. This means that the values in your array must only range from 1 to 5. Using sorting and searching techniques, develop an algorithm that will count the votes and indicate the winning candidate.

PSEUDOCODE:

Creating & displaying 100 unsorted elements:

```
START
   DECLARE list[100] as array of integers
   DECLARE i as integer

   FOR i = 0 to max_size - 1
      DISPLAY list[i]
      IF (i + 1) MOD 10 == 0 THEN
         DISPLAY newline
      END IF
   END FOR
END
```

Count votes & declare winner:

```
FUNCTION countVotes(list[], dataFind)
   DECLARE count = 0
   FOR i = 0 to max_size - 1
      IF list[i] == dataFind THEN
         count = count + 1
      END IF
   END FOR
   RETURN count
END FUNCTION

FUNCTION findWinner(list[])
   DECLARE candidates[5]

   FOR i = 0 to 4
```

		candidates[i] = countVotes(list, i + 1) END FOR DECLARE highVotes = candidates[0] DECLARE winner = 0 FOR i = 0 to 4 IF candidates[i] > highVotes THEN highVotes = candidates[i] winner = i + 1 END IF END FOR DISPLAY "Winner: Candidate " + winner END FUNCTION

CODE:	**Functions**
	
	`const int max_size = 100;`
	`void display(int list[]){`
	`for(int i = 0; i < max_size; i++ ){`
	`std::cout << list[i] << " ";`
	`if((i+1) % 10 == 0){`
	`std::cout << std::endl;`
	`}`
	`}`
	`}`
	`int countVotes(int list[], int dataFind){`
	`int count = 0;`
	`for(int i = 0; i < max_size; i++){`
	`if(list[i] == dataFind){`
	`count++;`
	`}`
	`}`
	`return count;`
	`}`
	`void findWinner(int list[]){`
	`int candidates[5];`
	
	`for(int i = 0; i < 5; i++){`
	`candidates[i] = countVotes(list, (i + 1));`
	`}`
	`int highVotes = candidates[0];`
	`int winner = 0;`
	`for(int i = 0; i < 5; i++){`
	`if(candidates[i] > highVotes){`
	`highVotes = candidates[i];`
	`winner = (i + 1);`
	`}`
	`}`
	`std::cout << "Winner: Candidate " << winner << std::endl;`
	`}`	

	Main Driver
	`int main(){`
	`  int randArr[max_size];`
	`  srand(time(0));`
	`  for(int i = 0; i < max_size; i++){`
	`  randArr[i] = rand()%5 + 1;`
	`  }`
	`  display(randArr);`
	`  std::cout << "Insertion Sort: \n";`
	`  insertionSort(randArr, max_size);`
	`  display(randArr);`
	`  std::cout << std::endl;`
	`  std::cout << "Votes for Candidate 1: " << countVotes(randArr, 1) << std::endl;`
	`  std::cout << "Votes for Candidate 2: " << countVotes(randArr, 2) << std::endl;`
	`  std::cout << "Votes for Candidate 3: " << countVotes(randArr, 3) << std::endl;`
	`  std::cout << "Votes for Candidate 4: " << countVotes(randArr, 4) << std::endl;`
	`  std::cout << "Votes for Candidate 5: " << countVotes(randArr, 5) << std::endl;`
	`  findWinner(randArr);`
	`  return 0;`
	`}`

OUTPUT TESTING:	`5 3 2 5 3 4 3 4 5 5`
	`3 4 1 4 3 3 2 3 4 2`
	`4 2 3 2 2 1 1 4 2 1`
	`5 2 1 2 2 5 2 2 5 5`
	`4 1 1 3 5 2 1 2 5 3`
	`2 1 5 2 3 5 2 2 3 2`
	`3 5 2 1 3 3 4 2 2 2`
	`5 4 1 2 1 1 5 3 5 4`
	`2 2 4 2 2 4 5 3 1 3`
	`1 2 2 2 4 4 3 5 1 1`

OUTPUT CONSOLE SHOWING SORTED ARRAY	MANUAL COUNT	COUNT RESULT OF ALGORITHM

`Insertion Sort:`
`1 1 1 1 1 1 1 1 1 1`
`1 1 1 1 1 1 1 2 2 2`
`2 2 2 2 2 2 2 2 2 2`
`2 2 2 2 2 2 2 2 2 2`
`2 2 2 2 2 2 2 2 3 3`
`3 3 3 3 3 3 3 3 3 3`
`3 3 3 3 3 3 3 4 4 4`
`4 4 4 4 4 4 4 4 4 4`
`4 4 5 5 5 5 5 5 5 5`
`5 5 5 5 5 5 5 5 5 5`
	Candidate 1 = 17
	Candidate 2 = 31
	Candidate 3 = 19
	Candidate 4 = 15
	Candidate 5 = 18
	Total = 100
	
	Winner = Candidate 2
		`Votes for Candidate 1: 17`
		`Votes for Candidate 2: 31`
		`Votes for Candidate 3: 19`
		`Votes for Candidate 4: 15`
		`Votes for Candidate 5: 18`
		`Winner: Candidate 2`

JUSTIFICATION FOR USING INSERTION SORT
Insertion sort seems like the best option because it doesn't do unnecessary comparisons or shifts once it finds the correct position for an element, which makes it slightly faster in this case, as shown by it needing only 99 passes compared to 100 for the other two algorithms. This is because insertion sort assumes the first element is already sorted by itself, so it starts comparing from the second element onward, skipping one unnecessary pass. In terms of time complexity, all three algorithms are $O(n^2)$ in the worst case, but insertion sort can perform closer to O(n) when the data is already mostly sorted, since it avoids unnecessary shifts. Since the vote array only contains a small range of repeated values (1 to 5), this made insertion sort a more efficient choice for sorting this specific data compared to selection sort, which always scans the remaining unsorted portion regardless of how sorted the data already is.

QUESTION:	Was your developed vote counting algorithm effective? Why or why not?
ANSWER:	It was effective, it did its job of counting. Although, I think it could be better. I could find the first occurrence of that number and know its index, then find the last occurrence of that number and just subtract their indexes. This works because the array is already sorted and the worst-case is going through the entire array. So I'm going to try and recreate it, even if I already made one because I have time.

PSEUDOCODE:	**Finding the first occurrence of the value looking for:**
	FUNCTION findFirst(list[], dataFind) FOR i = 0 to max_size - 1 IF list[i] == dataFind THEN RETURN i END IF END FOR RETURN -1 END FUNCTION
	Finding the last occurrence of the value looking for:
	FUNCTION findLast(list[], dataFind) FOR i = 0 to max_size - 1 IF list[i] > dataFind THEN RETURN i - 1 END IF IF list[i] == dataFind AND i == max_size - 1 THEN RETURN i END IF END FOR RETURN -1 END FUNCTION

Finding the number of occurrences using findFirst and findLast:

```
FUNCTION findCount(list[], dataFind)
   DECLARE first = findFirst(list, dataFind)
   IF first == -1 THEN
      RETURN 0
   END IF

   DECLARE last = findLast(list, dataFind)
   RETURN (last - first) + 1
END FUNCTION
```

Finding the winner using findCount:

```
FUNCTION findWinner1(list[])
   DECLARE candidates[5]

   FOR i = 0 to 4
      candidates[i] = findCount(list, i + 1)
   END FOR

   DECLARE highVotes = candidates[0]
   DECLARE winner = 0

   FOR i = 0 to 4
      IF candidates[i] > highVotes THEN
         highVotes = candidates[i]
         winner = i + 1
      END IF
   END FOR

   DISPLAY "Winner: Candidate " + winner
END FUNCTION
```

CODE:

```
int findFirst(int list[], int dataFind){
    for(int i = 0; i < max_size; i++){
        if(list[i] == dataFind) return i;
    }
    return -1;
}
int findLast(int list[], int dataFind){
    for(int i = 0; i < max_size; i++){
        if(list[i] > dataFind) return i - 1;
        if(list[i] == dataFind && i == max_size - 1) return i;
    }
    return -1;
}
int findCount(int list[], int dataFind){
    int first = findFirst(list, dataFind);
    if(first == -1) return 0;
    int last = findLast(list, dataFind);
    return (last - first) + 1;
}
void findWinner1(int list[]){
    int candidates[5];
    for(int i = 0; i < 5; i++){
        candidates[i] = findCount(list, (i + 1));
    }
    int highVotes = candidates[0];
    int winner = 0;
    for(int i = 0; i < 5; i++){
        if(candidates[i] > highVotes){
            highVotes = candidates[i];
            winner = (i + 1);
        }
    }
    std::cout << "Winner: Candidate " << winner << std::endl;
}
```

QUESTION:	Was your developed vote counting algorithm effective? Why or why not?
ANSWER:	This time, my developed vote counting algorithm is effective and efficient. This is because I only continue searching until the last occurrence of the value I'm looking for, taking advantage of the fact that the array is sorted. Since the array is sorted, once a value greater than the one I'm looking for is found, I know the search can stop there, instead of going through the rest of the list unnecessarily. This means the algorithm stops as soon as it reaches the last occurrence, rather than checking every remaining element.

CONCLUSION

I think learning about how each sorting technique is implemented helped visualize and understand how each of them works. How they were named also played a part in how they work. While they all seemed to do one job, compare and swap, they were still different in how they do it. Implementing the counting of passes into each sorting technique helped me understand how long they would take. By having randomized elements, we get to test them fairly, because each value is different every time.

Throughout the procedure, generating one randomized array of 100 elements and using it for bubble, selection, and insertion sort helped make the comparison fair since all three algorithms started with the same data. Bubble sort repeatedly compared and swapped elements until the larger values moved to their correct positions, while selection sort searched for the smallest remaining value and swapped it into place. Insertion sort was different because it builds a sorted section by shifting elements into their correct position when needed. This difference could be seen when comparing the number of passes each algorithm needed. Insertion sort completed sorting in 99 passes, one less than bubble and selection sort, because it treats the first element as already sorted and starts from the second element. This helped me understand that even though the three algorithms are all used for sorting, the way they work can still affect how efficient they are.

The supplementary activity was different from the main procedure because instead of arranging the values, the goal was to find how many times each value appeared in the array. Since the array only contained values from 1 to 5, sorting it first made it easier to search for each value. At first, I considered using binary search to find the values, but I realized that I could instead find the first and last occurrence of each candidate's value in the sorted array. By getting the difference between these two positions, I could determine how many times the value appeared without checking every element individually. This helped me understand how sorting the data first can make other operations, such as searching and counting, easier and more efficient.

Overall, I think I did well. I understood how each sorting technique works, although if I were to write them from scratch, I would need a little more guidance. This means I need to familiarize myself more with them so that I could ingrain the concept and become more natural at implementing them. I guess I could improve on explaining and understanding how each loop works and what their time complexities are.

COMPUTER ENGINEERING

QUEZON CITY

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **ClaudeAI, ChatGPT**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Grammar correction, summarization, and code assistance**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Used AI to articulate my thoughts, correct any grammar mistakes and flow, and helped me spot and understand code errors.**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.